The MABEL Tip-Over Safety Motion Model:
Lift-Aware Speed/Deceleration Envelopes and the Coupled ZMP Limiter (`mabel.motion_model`)

MABEL Project — Control & Teleoperation

November 2, 2026

Abstract

Every base velocity command in the MABEL stack — joystick, iOS/VisionPro nav sticks, web GUI, ROS navigation, or the admittance glide — must pass through one function, `safe_base_step`, before it reaches the swerve inverse kinematics. This document derives that function: the per-direction tip-over acceleration/speed caps and their $1/h$ dependence on the lift height, the coupled zero-moment-point (ZMP) criterion for combined translation + rotation, the degree-2 homogeneity argument that gives a *closed-form* projection of a steady twist into the safe envelope, and the transient acceleration clamp. We report the measured constants (CoM levers, the linear CoM-height fit and its ≤ 1.5 cm error), the validation evidence (a 450 000-step random sweep with zero contract violations; physical MuJoCo manoeuvres at 5.7° peak tilt where the unprotected base falls at 69°+), and one honestly-documented limitation: a liveness boundary for artificially injected far-over-limit states. The module is NumPy-only by design so the ROS navigation node imports it on the robot without MuJoCo.

Keywords: tip-over stability, zero-moment point, mobile manipulation, motion model, teleoperation safety.

1 Overview and design position

MABEL is a 62 kg mobile manipulator on a three-wheel (triangular) swerve base with a 0.635 m cascaded lift. Raising the lift raises the centre of mass (CoM), so the base's safe acceleration — and therefore safe speed — shrinks with lift height. The companion study `wbc_stability.pdf` shows that for MABEL's mass distribution the *dominant and essentially only* tip risk is dynamic base acceleration (static CoM shift needs a ∼14–18 kg hand payload to matter), which makes this motion model the robot's primary anti-tip-over defense.

The subpackage has three modules:

- `tip_over.py` — the safety envelope and `safe_base_step` (this document's subject). NumPy-only; the constants are *measured from the canonical MJCF* and re-derived by `experiments/wbc_stability.py` (E4). `mabel.config` re-exports everything, so `from mabel.config import safe_base_step` works everywhere.
- `com_support.py` — live measurement of the true CoM, the wheel-contact support polygon and the support margin from a running MuJoCo model; used to verify and re-fit the constants baked into `tip_over.py`.
- `base_response.py` — the system-identified *forward* model of the base (§5), used by ROS command-odometry.

The rule: never call `whole_body_controller.base_controller.swerve_ik` on a raw twist. Always route it through `safe_base_step` first.

2 Per-direction tip condition

Model the robot as a rigid body of mass m with CoM at height h braking (or accelerating) at $|a|$ on a flat floor. In the decelerating frame the inertial force ma acts at the CoM; the robot begins to tip about a support edge at horizontal lever distance d (CoM ground-projection to that edge) when the overturning moment exceeds the restoring moment:

$$m\,a\,h \;\le\; m\,g\,d \quad\Longrightarrow\quad a_{\text{tip}}(h) \;=\; \frac{g\,d}{h}, \tag{1}$$

mass cancels. The safe cap divides by a per-direction safety factor S:

$$a^{(k)}_{\max}(z) = \frac{g\,d_k}{S_k\,h(z)}, \qquad k \in \{\text{fwd},\ \text{back},\ \text{lat}\}. \tag{2}$$

MABEL's triangular base (single *rear* wheel) with a slightly forward CoM makes the *front* edge binding: the measured levers are $d_{\text{fwd}} = 0.078\,\text{m}$, $d_{\text{lat}} = 0.108\,\text{m}$, $d_{\text{back}} = 0.241\,\text{m}$ (re-measured 2026-06 after the lift-base drop). Forward braking keeps a conservative $S_{\text{fwd}} = 2.0$; lateral/rearward run agile at $S = 1.2$ (more margin *and* less dangerous).

CoM height vs. lift. The lift raises the CoM approximately linearly. The fit, re-measured from settled MuJoCo states,

$$h(z) \;=\; z_0 + k\,z, \qquad z_0 = 0.369\,\text{m},\ k = 0.451, \tag{3}$$

has a maximum error of 1.5 cm over the full travel (E4 of `wbc_stability.py`); the worst case *under*-estimates h at $z = 0$, over-estimating $a_{\max}$ by $<4\%$ — comfortably inside the 1.2–2.0 safety factors.

Speed caps. A speed cap is the speed a full stop from which takes a nominal stop time: $v^{(k)}_{\max} = a^{(k)}_{\max} \cdot T_k$ with $T_{\text{fwd}} = 0.80\,\text{s}$ (long, gentle) and $T_{\text{agile}} = 0.55\,\text{s}$ (snappy reversals), clamped to $[\,v_{\text{floor}},\ v^{(k)}_{\text{ceil}}\,]$ so the robot can always at least crawl (0.15 m/s) fully raised. Fig. 1 plots Eq. (2) and the resulting caps: forward speed falls from 0.80 m/s at $z = 0$ to 0.47 m/s fully raised (decel $1.04 \to 0.58\,\text{m/s}^2$).

3 Coupled motion: the ZMP criterion

The per-axis caps are the pure-axis special case. Under combined translation + rotation the CoM picks up Coriolis, centripetal and tangential accelerations, so a turn while driving can tip the robot at "safe" per-axis speeds. In the body frame, with CoM ground offset $r_c = (r_x, r_y)$ (measured: $(-0.076, -0.001)$ m), body twist $\xi = (v_x, v_y, \omega)$ and its rate $(a_x, a_y, \dot\omega)$, the horizontal CoM acceleration is

$$a_c \;=\; \begin{pmatrix} a_x - \omega v_y - \dot\omega\, r_y - \omega^2 r_x \\ a_y + \omega v_x + \dot\omega\, r_x - \omega^2 r_y \end{pmatrix}, \tag{4}$$

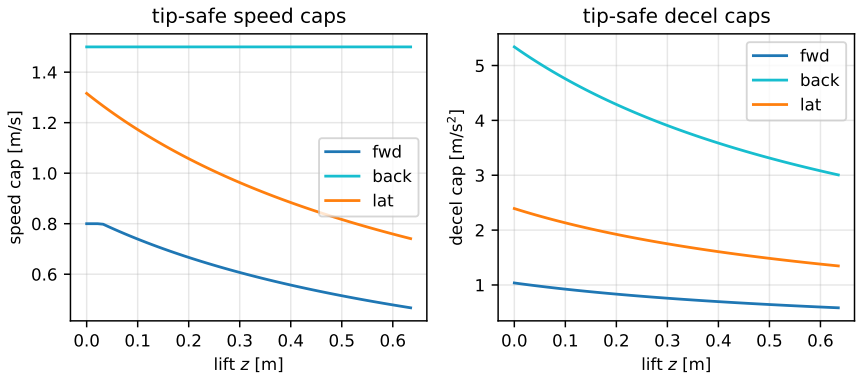


Figure 1: Tip-safe speed (left) and deceleration (right) caps vs. lift height, per direction (Eq. (2); generated by `experiments/doc_module_figures.py`).

(transport/Coriolis + tangential + centripetal). The zero-moment point $x_{\text{zmp}} = x_{\text{com}} - (h/g)\, a_c$ must stay inside the wheel-contact triangle [1, 2]. Equivalently, for each support edge k with outward unit normal o_k and CoM-to-edge lever d_k,

$$(-o_k) \cdot a_c \;\le\; \frac{g\, d_k}{S_k\, h(z)}. \tag{5}$$

We define the *usage*

$$u(\xi, \dot{\xi}, z) \;=\; \max_k \frac{\big[(-o_k) \cdot a_c\big]_+}{g\, d_k / (S_k\, h(z))}, \tag{6}$$

so $u \le 1$ is tip-safe. Fig. 2 maps the steady usage over (v_x, ω): the safe region visibly shrinks as the lift rises.

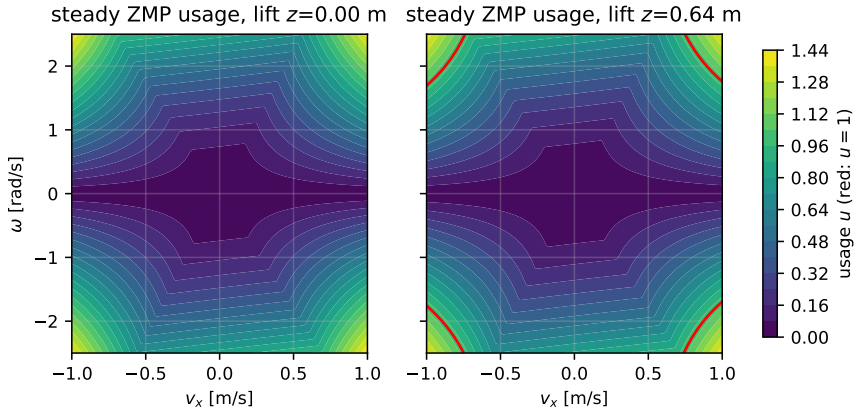


Figure 2: Steady ZMP usage (Eq. (6) at zero command acceleration) over forward speed and yaw rate, at $z = 0$ (left) and full lift (right). Red contour: $u = 1$.

3.1 Closed-form steady projection (homogeneity)

At zero *command* acceleration the steady CoM acceleration $a_c = (-\omega v_y - \omega^2 r_x,\ \omega v_x - \omega^2 r_y)$ is degree-2 homogeneous in the twist: $a_c(s\,\xi) = s^2 a_c(\xi)$. Usage (Eq. (6)) is positively linear in a_c,

hence

$$u(s\,\xi) = s^2\,u(\xi) \quad \Longrightarrow \quad s^\star = \min\bigl(1,\ u(\xi)^{-1/2}\bigr) \tag{7}$$

projects any steady twist into the envelope *in closed form* (`tip_speed_scale`). Two properties matter for safety: $u(0) = 0 \Rightarrow s^\star = 1$, so a **stop command is never throttled**; and an over-limit *target* is scaled back in rather than rejected, so the commanded set-point always lies inside the envelope. This replaced an earlier blend that dead-locked (held velocity) when the current twist already violated the constraint. The property $u(s\xi) = s^2 u(\xi)$ is verified to 10^{-9} relative accuracy over 200 random twists in `tests/test_motion_model.py`.

3.2 Transient clamp

The per-axis ramp toward the projected target can still create an unsafe *transient* a_c (accelerating while turning). `accel_safe_scale` scales the command acceleration $(a_x, a_y, \dot\omega)$ by $\alpha_\ell \in [0, 1]$ such that

$$u\bigl(\xi,\ \alpha_\ell\,\dot\xi,\ z\bigr)\ \le\ \max\bigl(1,\ u_0\bigr), \qquad u_0 = u(\xi, 0, z), \tag{8}$$

found by 12 bisection iterations. The cap $\max(1, u_0)$ makes $\alpha_\ell = 0$ always feasible and passes any usage-*reducing* (braking) command through unscaled.

3.3 The full step

`safe_base_step`$(\xi, \xi_{\text{tgt}}, \Delta t, z)$ chains: (1) clip ξ_{tgt} to the per-axis speed caps (Eq. (2)$\cdot T_k$); (2) project it into the steady envelope (Eq. (7)); (3) ramp each axis toward it at its direction-appropriate acceleration cap; (4) clamp the resulting transient with Eq. (8). The same function runs in the teleop smoother, the navigation stack, the admittance base and the validation harness, so all exercise identical logic.

4 Live CoM / support measurement (`com_support.py`)

The constants above are *baked*; `com_support.py` measures the truth at runtime from a MuJoCo state: the robot subtree CoM (`robot_com`), the support polygon from the actual wheel–floor contacts (`support_polygon`), the signed support margin (distance of the CoM projection to the nearest edge, negative outside), an angular tip margin, and `live_tip_usage` (Eq. (6) with the *live* CoM and polygon instead of the constants). E4 of `wbc_stability.py` compares fixed vs. live: XY bias 0.2 cm at home, height-fit error ≤ 1.5 cm — if a model change pushes these past ~ 1 cm XY, re-paste the printed constants into `tip_over.py`.

5 Calibrated base response (`base_response.py`)

System identification (`experiments/motion_id.py`, full robot on a bare floor through the package's own swerve IK) found the base's command response is strikingly clean inside the envelope:

$$\xi_{\text{actual}}\ =\ G\ \cdot\ \text{clamp}\bigl(\xi_{\text{cmd}},\ \text{envelope}\bigr), \qquad G = 0.989, \tag{9}$$

one uniform 1.1% slip gain on all three axes, fully decoupled (cross-terms <0.5%), linear with no deadband down to 0.05 m/s. Command *changes* ramp with a first-order time constant $\tau = 0.13$ s (the steer-and-choke transient, § swerve in `whole_body_controller.pdf`); the stateful `BaseMotionModel` includes it, the stateless `predict_twist`/`integrate` assume held commands. As a

process model it dead-reckons to ∼1% in free space and, because it integrates the *command*, it is immune to wheel spin — disagreement against wheel encoders (encoders ≫ model) is a slip/stuck detector.

6 Experiments

Numbers from `experiments/doc_module_figures.py` (figures here), `experiments/exp_wbc_validation.py` (V1, V2) and `experiments/wbc_stability.py` (E1–E4); all reproducible from scratch.

claim	test	result
per-axis caps monotone in lift	V1 (analytic sweep)	pass (fwd 0.80 → 0.47 m/s)
accel-clamp contract $u \le \max(1, u_0)$	V1, 450k random steps	0 violations
teleop envelope $\lvert v\rvert \le 0.8$, $\lvert\omega\rvert \le 1.5$	V1 sweep	max $u = 1.069$
stop from over-limit $(0.5, 0.8, 1.8)$ @ full lift	V1 liveness	stopped in 0.74 s
combined manoeuvre, physical MuJoCo @ z=0.30	V2	tilt 5.7° (raw: 69.3°, falls)
combined manoeuvre @ full lift	V2	tilt 3.2° (raw: 80.1°)
worst adversarial transient replay	V2	tilt 4.2°
fixed-CoM XY bias / height-fit error	E4	0.2 cm / 1.5 cm
homogeneity $u(s\xi) = s^2 u(\xi)$	pytest, 200 random	10^{-9} rel.
stop never throttled; braking passes unscaled	pytest	pass

7 Per-module actuator model (`swerve_modules.py`)

`base_response` is a body-level filter fitted to the MJCF; it has no wheels and so cannot say whether a motor is inside its limits, nor how non-holonomic the base is while a module re-steers. `SwerveModules` realises a commanded twist module by module: rigid-body module velocities, whole-twist desaturation to a safe wheel speed, a rate-limited first-order steer (shortest path, flip past 90°), the controller's own drive choke (0 past 0.52 rad of steer error, $\cos e$ inside it), a torque-derived wheel-acceleration cap, and a least-squares forward kinematics whose residual is scrub. Limits: drive 1.2 m/s safe (80 % of the measured SPARK loop collapse at 1.5 m/s; free speed 4.81 m/s), 5.5 N m (50 % of the MJCF's 11 N m) ⇒ 7.0 m/s^2; steer at full speed, which on the hardware is current-limited to a 90° turn in ∼170 ms (Ω_s = 10 rad/s, τ_s = 20 ms; measured on NOVA's identical MAXSwerve modules). Profiles: `mabel`, `mabel_mjcf` (the faster MJCF steer servo), `nova` (NOVA's driver). Derivation, sources and the measurements on a live Vision Pro take are in `web_gui/retarget_lab/docs/retarget_lab.pdf` (§Swerve modules, E5d); tests in `tests/test_swerve_modules.py`. It is used by the Retarget Lab and, through it, by NOVA's gateway; the robot's own command path is unchanged (`safe_base_step` → `swerve_ik`).

8 Implementation & usage

- NumPy-only import (asserted by `tests/test_imports_compat.py`): `from mabel.config import safe_base_step` costs no MuJoCo — this is what `mabel_ws/src/mabel_navigation` does on the Jetson.
- Per-tick pattern (every consumer):

```
vx, vy, w = safe_base_step(vx, vy, w, tvx, tvy, tw, dt, lift_z)
swerve_ik(vx, vy, w, data)
```

- Constants live in `mabel/motion_model/tip_over.py`; re-derive after any MJCF mass/geometry change with `wbc_stability.py` (E4).

9 Limitations

- **Liveness boundary for injected states.** For a state *far* outside the envelope (e.g. $(1.2, 1.0, 2.0)$ at full lift, $u_0 \gg 1$), every braking direction transiently raises the worst-edge usage, so Eq. (8) freezes the state instead of decelerating (`tests/test_motion_model.py` documents this as a strict `xfail`). Such states are unreachable *through* `safe_base_step` itself, but a lift rising under a fast twist shrinks the envelope toward this region. Candidate fix: a small ($\sim$2%) transient usage tolerance for strictly-decelerating commands; requires re-running V1/V2 before adoption.
- The CoM-height fit is linear (max error 1.5 cm); the constants go stale silently when the MJCF changes (mitigated by the E4 re-sync rule).
- Flat, level floor and rigid wheels are assumed; no payload term (a $\sim$14–18 kg hand payload shifts the static envelope, see `wbc_stability.pdf`).
- The friction limit (wheel slip before tip) is not modelled; on MABEL's rubber wheels the tip constraint binds first in every measured case.

References

[1] M. Vukobratović and B. Borovac, "Zero-moment point — thirty five years of its life," *Int. J. Humanoid Robotics*, 1(1), 2004.

[2] E. G. Papadopoulos and D. A. Rey, "A new measure of tipover stability margin for mobile manipulators," *Proc. IEEE ICRA*, 1996.

[3] Y. Cheng et al., "HoMMI: Learning whole-body mobile manipulation from human demonstrations" (CoM-over-support cost comparison in `wbc_stability.pdf`).